UNIT-4

LECTURE - 1

Dynamic Memory Management

When the number of data items keeps changing during the program execution and it (the data) increases, the memory allocates space to accommodate the additional data items. Such situtations in programming require dynamic memory management techniques.

Two ways to allocate memory -

- Where we know the exact size and storage at compile time.
- Where we allocate memory at run time and memory allocation is done dynamically.

With the help of Dynamic memory management we can allocate storage at run time. On the other hand we can de-allocate memory with the help of it.

In a C++ Program memory is devided into two parts-

Stack - All variables declared inside any function takes the stack's memory.

Heap - It is the unuased memory of the program which can be used at run time. The region of free memory from which memory is allocated is called the heap.

All C++ compilers will include at least these four dynamic allocation functions: calloc( ), malloc( ), free( ), realloc( ).

calloc

The calloc( ) function allocates memory, the size of which is equal to num * size. Means, calloc( ) allocates sufficient memory for an array of num objects of size size. The calloc( ) function returns a pointer to the first byte of the allocated region. If there is not enough memory to satisfy the request, a null pointer is returned. It is always important to verify that the return value is not null before attempting to use it.

MITRC

free

The free( ) function returns the memory pointed to by ptr to the heap. This makes the memory available for future allocation. It is imperative that free( ) only be called with a pointer that was previously allocated using one of the dynamic allocation system's functions (either malloc( ) or calloc( )).

malloc

The malloc( ) function returns a pointer to the first byte of a region of memory of size size that has been allocated from the heap. If there is insufficient memory in the heap to satisfy the request, malloc( ) returns a null pointer. It is always important to verify that the return value is not null before attempting to use it. Attempting to use a null pointer will usually result in a system crash.

realloc

The realloc( ) function changes the size of the previously allocated memory pointed to by ptr to that specified by size. The value of size may be greater or less than the original. If ptr is null, realloc( ) simply allocates size bytes of memory and returns a pointer to it. If size is zero, the memory pointed to by ptr is freed. If there is not enough free memory in the heap to allocate size bytes, a null pointer is returned, and the original block is left unchanged.

C++ supports dynamic memory allocation functions, called calloc(), malloc( ), free( ) and realloc. These are included only for the sake of compatibility with C.

MITRC

LECTURE - 2

Dynamic Allocation Operators

C++ provides two dynamic allocation operators: **new** and **delete**. These operators are used to allocate and free memory at run time. Dynamic allocation is an important part of almost all programs. The **new** operator allocates memory and returns a pointer to the start of it. The **delete** operator frees memory previously allocated using new.

Syntex-

ptr_var = new type; //ptr_var-> Pointer Variable

delete ptr_var;

ptr_var is a pointer variable that receives a pointer to memory that is large enough to hold an item of type type.

Example-1

```
#include <iostream>
using namespace std;

int main()
{
   double *ptr_variable = NULL;
   ptr_variable = new double;
   *ptr_variable = 3252.11;
   cout << "Value is : " << *ptr_variable << endl;
   delete ptr_variable;
   cout << "Value is : " << *ptr_variable << endl;
   return 0;
}
```

MITRC

Example-2

```
#include <iostream>
#include <new>
using namespace std;
int main()
{
   int *p;
   try
   {
      p = new int (55); // initialize to 55
   }
   catch (bad_alloc xa)
   {
      cout << "Allocation Failure\n";
      return 1;
   }
   cout << "At " << p << " ";
   cout << "is the value " << *p << "\n";
   delete p;
   cout << "At " << p << " ";
   cout << "is the value " << *p << "\n";
   return 0;
}
```

Allocating Arrays

We can allocate arrays using **new** by using this syntex:

ptr_var = new array_type [size];

MITRC

To free an array, we use this form of delete:

delete [ ] ptr_var;

Example-

```
#include <iostream>
#include <new>
using namespace std;
int main()
{
   int *p, i;
   try
   {
      p = new int [10]; // allocate 10 integer array
   }
   catch (bad_alloc xa)
   {
      cout << "Allocation Failure\n";
      return 1;
   }
   for(i=0; i<10; i++ )
      p[i] = i;
   for(i=0; i<10; i++)
      cout << p[i] << " ";
   delete [] p; // release the array
   return 0;
}
```

MITRC

Allocating Objects

We can allocate objects dynamically by using new. The dynamically created object acts just like any other object. When it is created, its constructor (if it has one) is called. When the object is freed, its destructor is executed.

Example-

```
#include <iostream>
#include <cstring>

using namespace std;
class score
{
   double cur_score;
   char name[80];
   public:
      void set(double n, char *s)
      {
         cur_score = n;
         strcpy(name, s);
      }
      void get_bal(double &n, char *s)
      {
         n = cur_score;
         strcpy(s, name);
      }
};

int main()
{
```

MITRC

```
    score *p;
    char s[50];
    double n;
    try
    {
       p = new score;
    }
    catch (bad_alloc xa)
    {
       cout << "Allocation Failure\n";
       return 1;
    }
    p->set(525.45, "Ajay");
    p->get_bal(n, s);
    cout << s << "'s Score is : " << n;
    cout << "\n";
    delete p;
    return 0;
}
```

MITRC

LECTURE - 3

Copy Constructor

A copy constructor is used to declare and initialize an object from another object. A copy constructor takes a reference to an object of the same class as itself as an argument. It defines how a copy of an object is made. The default copy constructor creates a bitwise copy of the object. The copy constructor executes when we pass an object as an argument to a function, making a local copy within the function. The copy constructor applies only to initializations.

For example, the statement:

myclass val2(val1); //It would define the object val2 and at the same time initialize it to the value of val1.

The process of initializing through a copy constructor is known as copy initialization.

The most common general form of a copy constructor is-

```
classname (const classname &obj)
{
        // body of constructor
}
```

Here, obj is a reference to the object on the right side of the initialization. It is permissible for a copy constructor to have additional parameters as long as they have default arguments defined for them.

Example-

```
#include <iostream>
using namespace std;

class myclass
{
   int *p;
```

MITRC

```
int size;
public:
  myclass(int sz)
  {
    try
    {
      p = new int[sz];
    }
    catch (bad_alloc xa)
    {
      cout << "Allocation Failure\n";
      exit(EXIT_FAILURE);
    }
    size = sz;
  }
  ~myclass()
  {
    delete [] p;
  }
  // copy constructor
  myclass(const myclass &a);

  void put(int i, int j)
  {
    if(i>=0 && i<size) p[i] = j;
  }

  int get(int i)
```

MITRC

```
    {
        return p[i];
    }
};

// Copy Constructor
myclass::myclass(const myclass &a)
{
  int i;
  try
  {
    p = new int[a.size];
  }
  catch (bad_alloc xa)
  {
    cout << "Allocation Failure\n";
    exit(EXIT_FAILURE);
  }
  for(i=0; i<a.size; i++)
    p[i] = a.p[i];
}

int main()
{
  myclass num(10);
  int i;
  for(i=0; i<10; i++)
    num.put(i, i);
```

MITRC

```
    for(i=9; i>=0; i--)
        cout << num.get(i);
    cout << "\n";
    // create another myclass and initialize with num
    myclass x(num); // invokes copy constructor
    for(i=0; i<10; i++)
        cout << x.get(i);
    return 0;
}
```

In above example, The copy constructor is called, memory for the new myclass is allocated and stored in x.p, and the contents of num are copied to x's myclass.

DESTRUCTORS:

A destructor, as the name implies, is used to destroy the objects that have been created by a constructor. Like a constructor, the destructor is a member function whose name is the same as the class name but is preceded by a tilde(~).

For example , the destructor for the class String can be defined as shown below:

~String( ) { }

A destructor never takes any argument nor does it return any value. It will be invoked implicitly be the compiler upon exit from the program to clean up storage that is no longer accessible. It is a good practice to declare destructors in a program since it releases memory space for future use. Whenever new is used to allocate memory in the constructors, we should use delete to free that memory.

MITRC

LECTURE - 4

Assignment Operator

We've used the assignment operator so many times. Suppose obj1 and obj2 are objects. Unless we tell the compiler otherwise, the statement-

obj2 = obj1; // set obj2 to the value of obj1

It will cause the compiler to copy the data from obj1, member by member, into obj2. This is the default action of the assignment operator, =.

Initializing an object with another object, as in-

myclass obj2(obj1); // initialize obj2 to the value of obj1

It causes a similar action. The compiler creates a new object, obj2, and copies the data from obj1, member by member, into obj2. This is the default action of the copy constructor.

Every C++ class object automatically has four special member functions: a default constructor, a default destructor, an assignment operator, and a copy constructor. The default assignment operator allows us to assign one object to another, as in object1 = object2. The copy constructor executes when we pass an object as an argument to a function, making a local copy within the function.

Overloading the Assignment Operator

```
#include <iostream>
using namespace std;

class alpha
{
   private:
   int data;
   public:
      alpha() //no-arg constructor
```


MITRC

```
    { }

    alpha(int d) //one-arg constructor
    { data = d; }

    void display() //display data
    { cout << data; }

    alpha operator =(alpha& a) //overloaded = operator
    {
       data = a.data; //not done automatically
       cout << "\nAssignment operator invoked";
       return alpha(data); //return copy of this alpha
    }
};

int main()
{
   alpha a1(65);
   alpha a2;
   a2 = a1; //invoke overloaded =
   cout << "\na2="; a2.display(); //display a2
   alpha a3 = a2; //does NOT invoke =
   cout << "\na3="; a3.display(); //display a3
   cout << endl;
   return 0;
}
```

In this example, in main(), we define a1 and give it the value 65, and define a2 but give it no value. Then we use the assignment operator to set a2 to the value of a1:

MITRC

a2 = a1; // assignment statement

This causes our overloaded operator=() function to be invoked. Here's the output from ASSIGN:

Assignment operator invoked

a2=65

a3=65

When we overload the assignment operator, we almost always want to overload the copy constructor as well (and vice versa). We don't want our custom copying routine used in some situations, and the default member-by-member scheme used in others.

MITRC

LECTURE - 5

Virtual Destructor

Base class destructors should always be virtual. Suppose we use delete with a base class pointer to a derived class object to destroy the derived-class object. If the base-class destructor is not virtual then delete, like a normal member function, calls the destructor for the base class, not the destructor for the derived class. This will cause only the base part of the object to be destroyed.

Example-1-

```
#include <iostream>
using namespace std;

class Base
{
   public:
      ~Base() //non-virtual destructor
      // virtual ~Base() //virtual destructor
      { cout << "Base Destroyed\n"; }
};

class Derv : public Base
{
   public:
      ~Derv()
      { cout << "Derv destroyed\n"; }
};

int main()
```

MITRC

```
{
   Base* pBase = new Derv;
   delete pBase;
   return 0;
}
```

Output-

Base Destroyed

This shows that the destructor for the Derv part of the object isn't called. In the listing the base class destructor is not virtual.

Example-2-

```
#include<iostream>
using namespace std;

class base
{
   public:
      base()
      {
         cout << "Constructing Base Class\n";
      }
      virtual ~base()
      {
         cout << "Destructing Base Class\n";
      }
};

class derived: public base
{
```

MITRC

```
   public:
      derived()
      {
         cout << "Constructing Derived Class\n";
      }
      ~derived()
      {
         cout << "Destructing Derived Class\n";
      }
};

int main()
{
   derived *deriv = new derived();
   base *bptr = deriv;
   delete bptr;
   return 0;
}
```

Output-

Constructing Base Class

Constructing Derived Class

Destructing Derived Class

Destructing Base Class

if none of the destructors has anything important to do (like deleting memory obtained with new) then virtual destructors aren't important. But in general, to ensure that derived-class objects are destroyed properly, we should make destructors in all base classes **virtual**.

MITRC